

1. Introduction

The requirement for this assignment is to design an agent capable of scanning supermarket bill contents, extracting relevant information, and answering users' related questions. The assignment specifies two key requirements:

- Can process **several** images simultaneously in a single operation.
- Can **reject irrelevant** questions.

2. Method and Solution

2.1. OCR Function

First and foremost, the key step is to scan the supermarket receipt image and feed it into the LLM. Here, directly use the two functions provided in the documentation—`image_to_base64` and `get_image_data_url`—to convert the JPG image into a Base64-encoded string, and construct a suitable input for GEMINI api call.

```
import base64
import mimetypes

# Helper function to read and encode image
def image_to_base64(img_path):
    with open(img_path, "rb") as img_file:
        return base64.b64encode(img_file.read()).decode('utf-8')

# Helper function to encode local file to Base64 Data URL
def get_image_data_url(image_path):
    # Guess the mime type (e.g., image/png, image/jpeg) based on file extension
    mime_type, _ = mimetypes.guess_type(image_path)
    if mime_type is None:
        mime_type = "image/png" # Default fallback

    encoded_string = image_to_base64(image_path)

    # Construct the Data URL
    return f"data:{mime_type};base64,{encoded_string}"
```

2.2. Prompt Engineering

To fulfill the requirement mentioned in the assignment—rejecting users' irrelevant questions—the simplest approach is to use prompt engineering by including a clear emphasis within the prompt itself.

```

from langchain_core.prompts import ChatPromptTemplate
prompt = ChatPromptTemplate.from_messages([
    ("system", """
    You are a helpful billing assistant. Your sole task is to assist users with
    questions related to bills, including interpreting the content of bills and
    checking for promotional offers or discounts listed on them.

    Important!!: However, you are quite limited—you can only answer questions
    directly related to bills. You will not respond to any other types of
    questions, and you may politely decline to answer unrelated inquiries.
    """),
    ("human", [
        {"type": "text", "text": "{question}"},
        {"type": "image_url", "image_url": {"url": "{image_url}"}}],
    )],
])

```

In the prompt, I explicitly instruct the LLM to answer only questions related to billing and not to respond to any other types of questions. If a user asks an unrelated question, the model should politely decline to answer.

2.3. Agent based on LangChain

Due to the relatively high barrier to using the Vertex API (I'm located in mainland China, where network access is inconvenient), I opted for a third-party platform—[AIHubMix](#). This platform supports most mainstream models available on the market and allows them to be invoked through a unified API interface, which is also the development platform I routinely use for Agent development.

To facilitate the teaching assistants' evaluation and testing of my assignment, I have included my API key directly in the .ipynb file. I've also performed a quick test, confirming that the Gemini model can indeed be successfully called and used as expected.

To facilitate the teaching assistants' evaluation of the assignment, you may temporarily use my API key. The usage remains the same: simply add `OPENAI_API_KEY` in the "Secrets" section of Colab's left-side panel.

My API key is: `sk-HtG6ITmDDgXh5hmmA54a3eB1CeF24487A427977fA8BaE9E5`

```

from google.colab import userdata
openai_api_key = userdata.get('OPENAI_API_KEY')

from langchain.chat_models import init_chat_model

llm = init_chat_model(
    "gemini-2.5-flash",
    temperature=0,
    model_provider="openai",
    base_url="https://aihubmix.com/v1",
    api_key=openai_api_key)

```

```

# result = llm.invoke("Hello world - What does CUHK stand for?")
result = llm.invoke("Hello, please introduce yourself. What model are you? What's your name?")
print(result.content)

```

... Hello!

I am a large language model, trained by Google.

My model name is `**Gemini**`.

I don't have a personal name, as I am an AI and not a human. You can just refer to me as an AI assistant or by my model name.

How can I help you today?

The next step is simply to chain together the prompt and the LLM, which will create a basic intelligent agent capable of recognizing supermarket receipts. I used an irrelevant question here for testing, and it can be observed that instead of answering me, it politely declined.

```
image_path = "/content/receipts/34aa8044-346a-4df0-a18a-596405f3f168.jpg"
image_data_url = get_image_data_url(image_path)
```

```
from langchain_core.prompts import ChatPromptTemplate
prompt = ChatPromptTemplate.from_messages([
    ("system", """
    You are a helpful billing assistant. Your sole task is to assist users with
    questions related to bills, including interpreting the content of bills and
    checking for promotional offers or discounts listed on them.

    Important!!: However, you are quite limited—you can only answer questions
    directly related to bills. You will not respond to any other types of
    questions, and you may politely decline to answer unrelated inquiries.
    """),
    ("human", [
        {"type": "text", "text": "{question}"},
        {"type": "image_url", "image_url": {"url": "{image_url}"}}
    ]),
])

chain = prompt | llm

response = chain.invoke({
    "question": "How is the weather today?",
    "image_url": image_data_url # Pass the full data URI here
})

print(response.content)
```

I am a billing assistant and can only answer questions related to bills. I cannot provide information about the weather.

3. Innovation

3.1. Multi-bill inquiry

To fulfill the requirement mentioned in the assignment—processing **several** supermarket receipt images—I have separated the prompt into two parts: the **System Prompt** and the **Human Prompt**. Based on the number of images provided by the user, I dynamically add the corresponding `image_url` fields.

```
SYSTEM_MESSAGE = """
```

```
You are a helpful billing assistant. Your sole task is to assist users with
questions related to bills, including interpreting the content of bills and
checking for promotional offers or discounts listed on them.
```

```
Important!!: However, you are quite limited—you can only answer questions
directly related to bills. You will not respond to any other types of
questions, and you may politely decline to answer unrelated inquiries.
"""
```

```
# Build dynamic human message content
```

```
human_message_content = [{"type": "text", "text": user_input}]
```

```
for url in image_data_urls:
```

```
    human_message_content.append({"type": "image_url", "image_url": {"url": url}})
```

```
prompt = ChatPromptTemplate.from_messages([
```

```
    ("system", SYSTEM_MESSAGE),
```

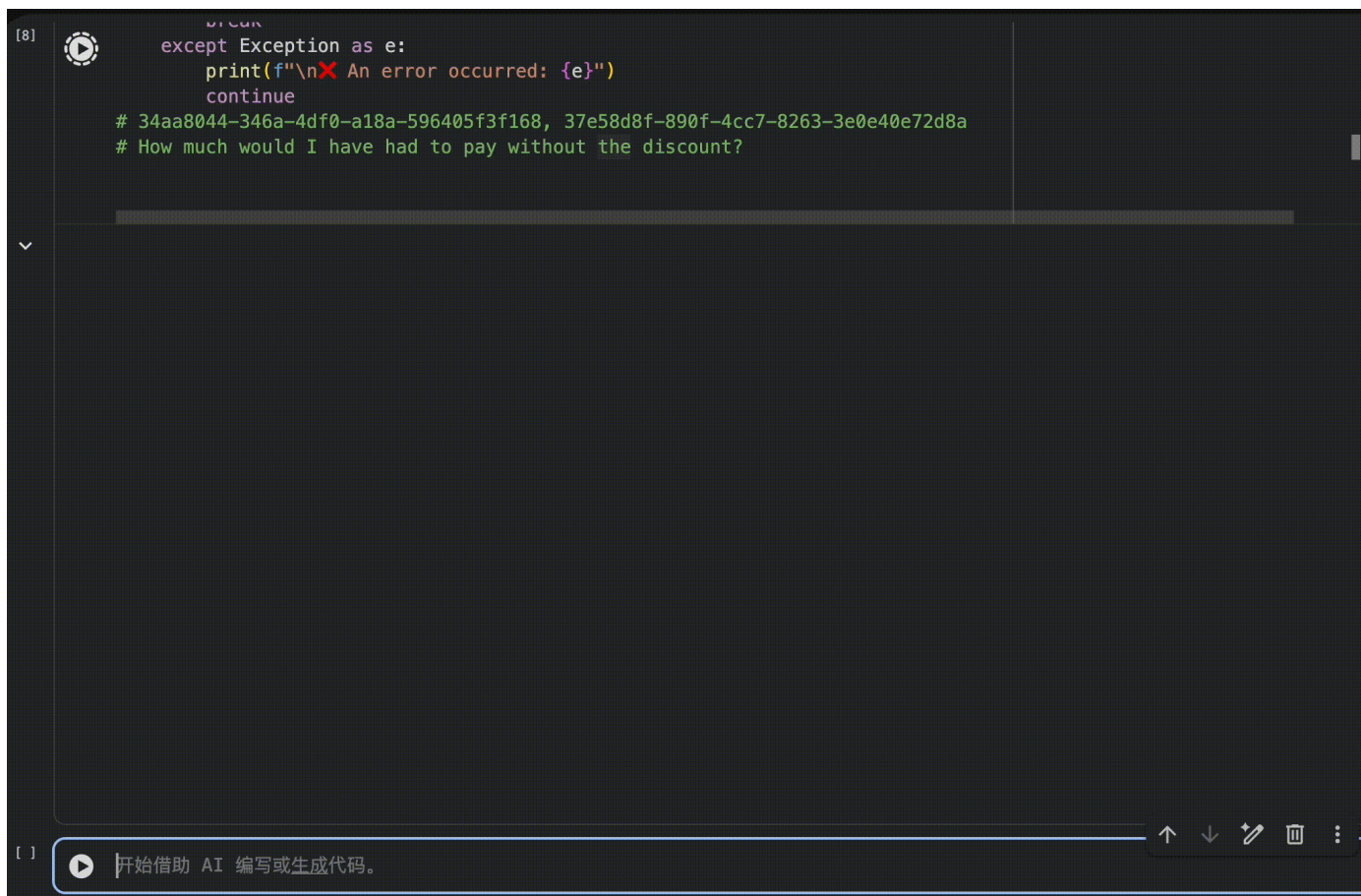
```
    ("human", human_message_content)
```

```
])
```

```
chain_bot = prompt | llm
```

3.2. Interactive conversational bot

To better simulate an interactive experience, I created an **interactive bot** and implemented **streaming output** to enhance the user experience.

A screenshot of a code editor with a dark theme. The editor shows a Python code snippet for an interactive bot. The code includes a try-except block to handle exceptions, a print statement with a red 'X' icon, and a continue statement. Below the code, there are two lines of green text: a long alphanumeric string and a question about a discount. The editor has a sidebar on the left with a file explorer icon and a bottom bar with a play button and the text '开始借助 AI 编写或生成代码。' (Start using AI to write or generate code.).

```
[8]  except Exception as e:  
    print(f"\n❌ An error occurred: {e}")  
    continue  
# 34aa8044-346a-4df0-a18a-596405f3f168, 37e58d8f-890f-4cc7-8263-3e0e40e72d8a  
# How much would I have had to pay without the discount?
```

[1]  开始借助 AI 编写或生成代码。

4. Conclusion

In summary, the key focus of this assignment lies in designing prompts that can **reject out-of-domain questions** and implementing dynamic prompts capable of **processing multiple images simultaneously**. Beyond fulfilling these basic functionalities, I also developed an **interactive conversational bot** that can handle supermarket receipts containing multiple images through dialogue, with support for **streaming output**.

This assignment has been submitted to GitHub: <https://github.com/cjsmt/FTEC5660>